

BigData Pipeline Project — Phase 3 Raporu

Airflow Orkestrasyonu ve dbt ile Medallion Mimarisi

Ahmet Berat Yıldırım

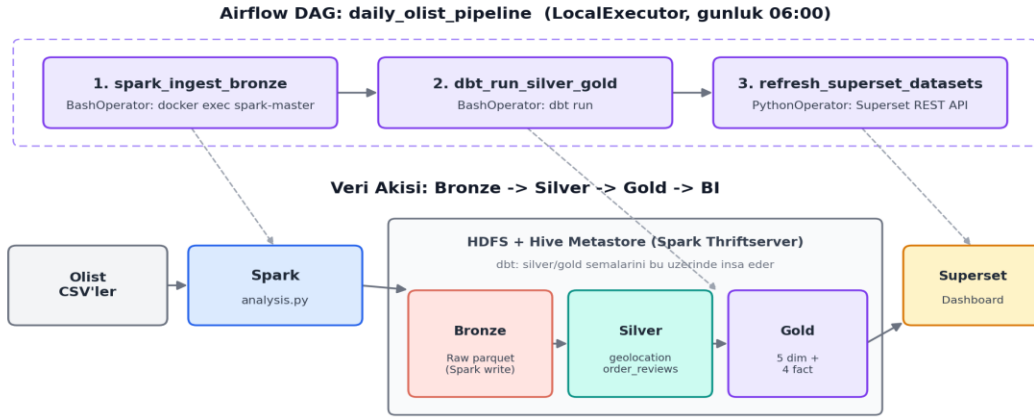
1. Genel Bakış

Phase 1-2'de Bronze/Silver/Gold hattı elle çalıştırılıyordu (docker exec ile Spark, sonra register_gold_tables.py, sonra Superset kontrolü). Phase 3'te bu süreç Apache Airflow ile otomatikleştirildi ve Silver/Gold dönüşüm mantığı PySpark script'inden (build_star_schema.py) modüler dbt SQL modellerine taşındı.

2. Mimari Kararlar

- Apache Doris yerine mevcut Hive Metastore + Spark Thriftserver — dbt-spark[pyhive] adaptörü zaten bu bağlantıyı destekliyor, yeni bir dağıtık sistem kurmaya gerek kalmadı.
- MinIO yerine HDFS — HDFS zaten Bronze veriyle dolu ve çalışıyordu, MinIO denenmiş ama hiç kullanılmamıştı.
- Airflow 3.3.0 (2.x Nisan 2026'da EOL oldu), LocalExecutor tek makinede tek DAG için Celery/Redis gereksiz kaynak tüketimi.
- register_gold_tables.py'nin Hive tablo oluşturma kısmı devre dışı artık bu işi dbt yapıyor; script'in Superset REST API mantığı DAG'ın 3. task'ına taşındı.

3. Sistem Mimarisi



Kesikli oklar: hangi Airflow task'ının hangi aşamayı tetiklediğini gösterir

Airflow'un üç görevi (üst) ve tetiklediği Bronze→Silver→Gold→Superset veri akışı (alt)

4. Uygulama

Repo yapısı (dal: phase3):

- docker/docker-compose-airflow.yml, Dockerfile.airflow — Airflow 3.3.0, custom image (docker CLI + dbt-core + dbt-spark)
- dbt_project/models/silver/ — silver_geolocation, silver_order_reviews

- dbt_project/models/gold/ — 5 dimension (dim_date, dim_customer, dim_seller, dim_product, dim_payment_type) + 4 fact (fact_order_items, fact_payments, fact_delivery, fact_reviews), + schema.yml (15 unique/not_null testi)
- dags/olist_pipeline_dag.py — daily_olist_pipeline DAG'ı

DAG'ın görev zinciri (operatör seçimleri kod içinde docstring olarak gerekçelendirildi):

```
spark_ingest_bronze = BashOperator(...) # docker exec spark-master spark-submit ...
dbt_run_silver_gold = BashOperator(...) # dbt run (dbt image içinde kurulu)
refresh_superset = PythonOperator(...) # Superset REST API ile dataset senkronizasyonu

spark_ingest_bronze >> dbt_run_silver_gold >> refresh_superset
```

Tüm kodun tamamı GitHub'daki phase3 dalında: [docker/](#), [dbt_project/](#), [dags/](#) klasörleri.

5. Karşılaşılan Zorluklar

PySpark'ın tek-seferlik script modelinden dbt + Thriftserver'ın çok-oturumlu servis modeline geçiş, önceki fazlarda görülmemiş dört sorun çıkardı:

- "Filesystem closed" — Hive'in encryption-zone kontrolü ile Hadoop'un FileSystem cache'i çakıştı. Çözüm: Thriftserver başlatma komutuna spark.hadoop.fs.hdfs.impl.disable.cache=true eklendi.
- Derby metastore sıfırlanması Thriftserver --force-recreate ile yeniden oluşturulunca Bronze tablo kayıtları da gitti (Phase 2'den bilinen kırılabilirlik). Çözüm: register_tables.py yeniden çalıştırıldı.
- "Managed table location already exists" önceki başarısız denemelerden kalan yarım HDFS klasörleri yeni CTAS ile çakıştı. Çözüm: klasör elle silindi.
- Phase 2 dashboard'ı bozuldu eski gold tabloları default şemasında silinmişti; dashboard'u yeniden kurmak yerine dataset'lerin şeması gold'a güncellendi, chart'lar değişmeden çalışmaya devam etti.

6. Airflow Core Bileşenleri

Airflow; DAG'ları tarayıp görev bağımlılık grafiğini kuran Scheduler, izleme/tetikleme sağlayan API Server, görevleri fiilen çalıştıran Executor (bu projede LocalExecutor) ve tüm durumun tutulduğu Metadata Veritabanı (Postgres) bileşenlerinden oluşur.

7. Sonuçlar

- dbt run: 11/11 model başarılı · dbt test: 15/15 test başarılı
- Airflow DAG: 3/3 görev başarılı, 0 başarısız çalıştırma
- Superset dashboard: 7/7 chart, gold şemasındaki güncel verilerle çalışıyor

Uçtan uca zincir doğrulandı: Olist CSV'leri → Spark (Bronze) → dbt (Silver/Gold) → Superset, tamamı tek bir Airflow DAG'ı tarafından tetiklenip izlenebiliyor.

8. DAG Walk-through: Sınırlar, Operatör Seçimleri, Kaynak Ayarları

Task	Sınır	Operatör + Gerekçe	Kaynak Ayarı
1. spark_ingest_bronze	Sadece ham CSV → Bronze Parquet dönüşümü. Hive katalog işlemi yapmaz, Silver/Gold'a dokunmaz.	BashOperator + docker exec spark-master. DockerOperator değil — amaç zaten çalışan spark-master servisine bağlanmak, yeni container/image ayağa kaldırmak değil.	Ayrı pool yok — LocalExecutor'da task'lar sıralı (>>) çalışıyor, spark-master üzerinde eşzamanlı kaynak çakışması riski yok.
2. dbt_run_silver_gold	Bronze'un güncel olduğunu varsayar (task 1'e bağımlı). Silver+Gold SQL modellerini çalıştırır. Superset'e dokunmaz.	BashOperator — dbt-core Airflow image'inin içine kurulu, ekstra exec/container katmanına gerek yok.	on-run-start hook'u ile spark.hadoop.fs.hdfs.impl.disable.cache her session başında zorlanıyor (Filesystem closed sorununa karşı).
3. refresh_superset_datasets	Sadece Superset REST API üzerinden dataset kaydı/metadata	PythonOperator — login + CSRF token + JSON payload akışı, requests.Session ile	—

Task	Sınır	Operatör + Gerekçe	Kaynak Ayarı
	senkronizasyonu. Hive/Spark işlemi ya da chart/dashboard oluşturma yapmaz — bilinçli bir sınır.	BashOperator+curl zincirinden daha güvenilir/okunur.	

DAG seviyesi kaynak ayarları: max_active_runs=1 (aynı anda iki run'ın HDFS/Hive'a eşzamanlı yazmasını önler — idempotent olmayan CTAS operasyonunda kritik), retries=1 + retry_delay=2 dakika (geçici kesintilere karşı tek otomatik deneme). execution_timeout şu an tanımlı değil; süreler öngörülebilir olduğu için gerek görülmedi, ileride eklenebilecek bir iyileştirme.

Medallion Mimarisinin Faydaları

- Veri kalitesi sınırları: Bronze hiç değiştirilmeden korunur, hata bulunduğunda ham veriye her zaman dönülebilir; Silver temizliği, Gold iş mantığını izole eder.
- Modülerlik: 16 KB'lık tek bir Python script yerine, her biri tek bir tabloyu tanımlayan 11 küçük .sql dosyası.
- Otomatik bağımlılık grafiği: dbt, ref() kullanımına göre modelleri doğru sırada kendisi çalıştırır; PySpark script'inde bu sıralama fonksiyon çağrı sırasıyla elle yönetiliyordu.
- Test edilebilirlik: schema.yml ile deklaratif unique/not_null testleri; orijinal script'te hiç otomatik test yoktu.
- Yeniden kullanılabilirlik: aynı Silver modeli birden fazla Gold modelinde ref() ile tekrar kullanılabilir, kod tekrarı azalıyor.
- Dokümantasyon potansiyeli: dbt docs generate ile bağımlılık grafiği otomatik bir lineage diyagramına dönüştürülebilir.